

Module 01: Foundations of Scalability & Architecture

1. What is System Design?

Imagine building a tiny wooden cabin versus a 100-story skyscraper. A cabin is simple: one room, basic materials, and easy to build. But if you try to stack 100 floors on top of that cabin without planning, it will instantly collapse!

In software, **System Design** is the blueprint for building that skyscraper. It is the process of defining the architecture, components, modules, interfaces, and data flow of a system so that it satisfies specified requirements smoothly and efficiently.

The Core Equation of System Design:

System Design = Architecture + Components + Communication + Data Flow + Trade-offs

Key Pillars Every System Designer Focuses On:

- **Scalability** — The ability to effortlessly grow as workload, users, and data increase.
- **Performance** — Delivering lightning-fast responses with minimal latency.
- **Availability** — Ensuring the application remains accessible 24/7 whenever users need it.
- **Reliability & Fault Tolerance** — Guaranteeing correct operation even when hardware or software components fail.
- **Maintainability** — Keeping the system modular so future engineers can easily update and enhance it.

2. Why Do We Need System Design?

When a student builds a project for a class assignment, it usually runs on a local computer with a single database. It works perfectly for 5 or 10 users. But what happens if 500,000 users try to access it at once on launch day?

The Real-World Dilemma:

Without proactive architecture, sudden surges in traffic cause **sluggish responses, server crashes, database deadlocks, and total downtime.**

- **Common Bottlenecks in Growing Systems:**
 - **Server Overload:** CPU maxes out at 100%, RAM gets exhausted, and incoming requests get dropped.
 - **Network Congestion:** Bandwidth limits saturate, causing immense latency.
 - **Database Bottlenecks:** Too many read/write queries lock disk operations, bringing everything to a grinding halt.

A simple application can initially be represented as:

Workflow:



3. Scalability: Vertical vs. Horizontal

Scalability is the system's ability to adapt and handle expanding workloads without degrading user experience. Think of it as expanding a highway: when traffic grows, you must give cars more lane capacity.

There are two primary scaling strategies:

- 1) Vertical Scaling (Scale Up)
- 2) Horizontal Scaling (Scale Out)

A) Vertical Scaling (Scale Up)

Vertical Scaling means making your existing single server bigger and stronger. You upgrade hardware by adding more CPU cores, extra RAM, or faster NVMe storage.

Real-World Analogy:

Buying a faster sports car (Ferrari) instead of a sedan. It goes much faster, but it still seats a limited number of people and hits physical limits.

Advantages

- Simple to implement and manage.

System Design Series

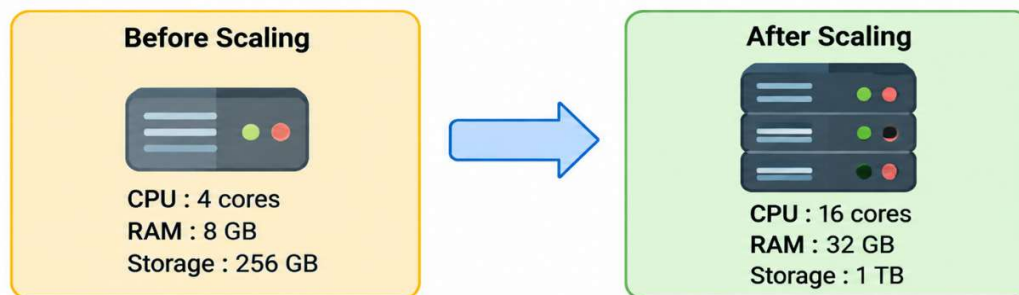
(Week 1)

- Requires fewer changes to the application architecture.
- Easier to maintain because the system can continue running on a single server.
- Suitable when the workload can be handled by a larger machine.

Pros & Cons of Vertical Scaling:

- **Pro:** Extremely simple to implement; requires zero code or architectural modifications.
- **Con (Hardware Limit):** Every server hits a ceiling; motherboard/RAM limits are finite.
- **Con (Downtime & Cost):** Upgrading hardware requires rebooting (downtime), and supercomputers get exponentially expensive.

Increasing the resources of an existing server



B) Horizontal Scaling (Scale Out)

Horizontal Scaling means adding more server instances to your system. Instead of upgrading one super-machine, you pool work across many commodity servers.

Real-World Analogy:

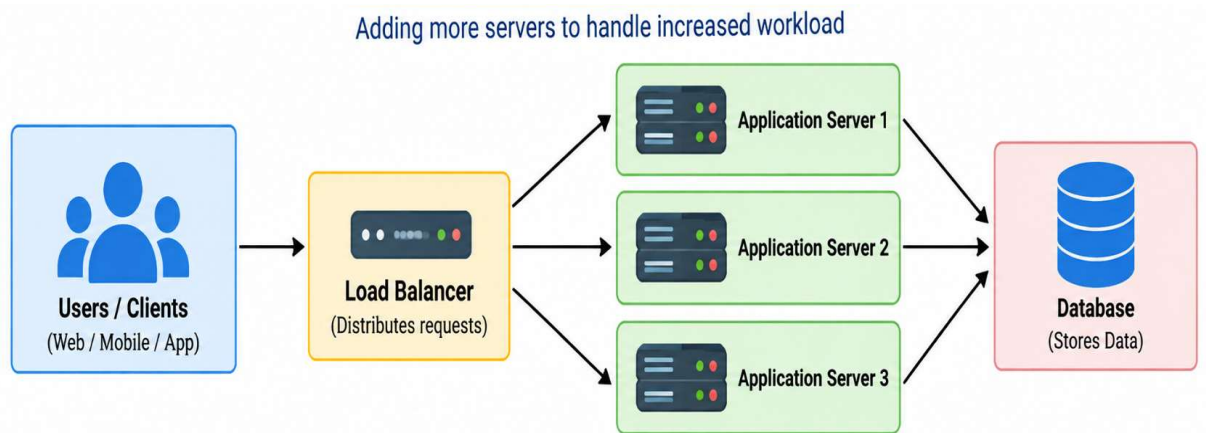
Hiring a fleet of 10 taxi drivers instead of buying one fast bus. If one taxi breaks down, 9 others keep working smoothly!

Advantages :

- Additional servers can be added as workload increases.
- Work can be distributed across multiple machines.
- Multiple application instances can provide additional capacity if one instance becomes unavailable.
- It provides a practical approach for systems that need to handle continuously increasing traffic.

Pros & Cons of Horizontal Scaling:

- **Pro (Unlimited Scale):** You can infinitely keep adding servers as user demand increases.
- **Pro (High Availability):** Zero single-point-of-failure; if Server 1 crashes, Server 2 and 3 carry the traffic.
- **Con (System Complexity):** Requires load balancing, stateless app designs, and network orchestration.
- Monitoring, networking, deployment, and failure handling become more involved.



Statelessness: The Secret to Horizontal Scaling

- **Stateful:** If Server A stores user login sessions in its local memory, user requests are "stuck" to Server A. If Server A dies, the user gets logged out.
- **Stateless:** Application servers store NO user session data locally. Sessions live in a shared central store (like Redis). Any request can land on ANY server seamlessly.

Example:

An online shopping application initially runs on one server:

Users → Application Server → Database

As the number of users increases, the architecture can evolve to:

Users → Load Balancer → Multiple Application Servers → Database

This change allows incoming requests to be distributed instead of depending entirely on a single application server.

4. Architecture Choices: Monolith vs. Microservices

- As an application grows, one of the important system-design decisions is how to organize its functionality.
- An application can be developed as a single unit, known as a **Monolithic Architecture**, or divided into smaller, independently focused services, known as **Microservices Architecture**.

Monolithic Architecture

What is a Monolithic?

- A Monolith packs all system components (User Management, Orders, Billing, Notifications) into a single unified codebase and deployment package.
- Although the application contains different modules, they are packaged and deployed together.

Example

Consider an e-commerce application containing:

- User Management
- Product Management
- Order Management
- Payment Processing

In a monolithic application, these functionalities can all be part of one application.

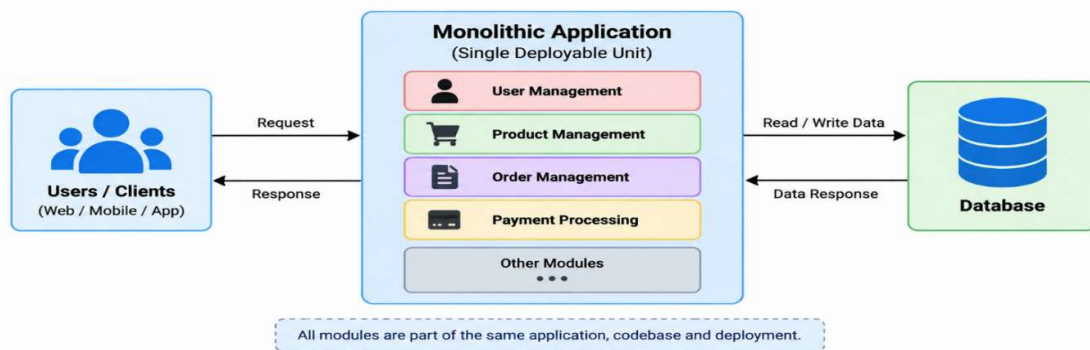


Monolith Trade-offs:

- **Advantages:** Extremely fast to build initially, easy end-to-end debugging, and simple single-step deployment.
- **Disadvantages:** Hard to scale individual features; a single bug in payment processing can crash the entire application.
- Simpler Communication
- Good Starting Point

Limitations :

- Growing Codebase
- Coupled Deployment
- Limited Independent Scaling Increasing Complexity



Microservices Architecture

- Microservices break down an application into independent, decoupled services (e.g., Auth Service, Order Service, Payment Service). Each service has its own dedicated database and communicates over lightweight API calls.
- Each service focuses on a specific business capability.
- Services communicate with each other through network-based communication such as APIs.
- Individual services can be developed, deployed, and scaled independently.

Example

The same e-commerce application can be divided into:

- **User Service** → Manages users and authentication.
- **Product Service** → Manages products and inventory.
- **Order Service** → Handles customer orders.
- **Payment Service** → Processes payments.



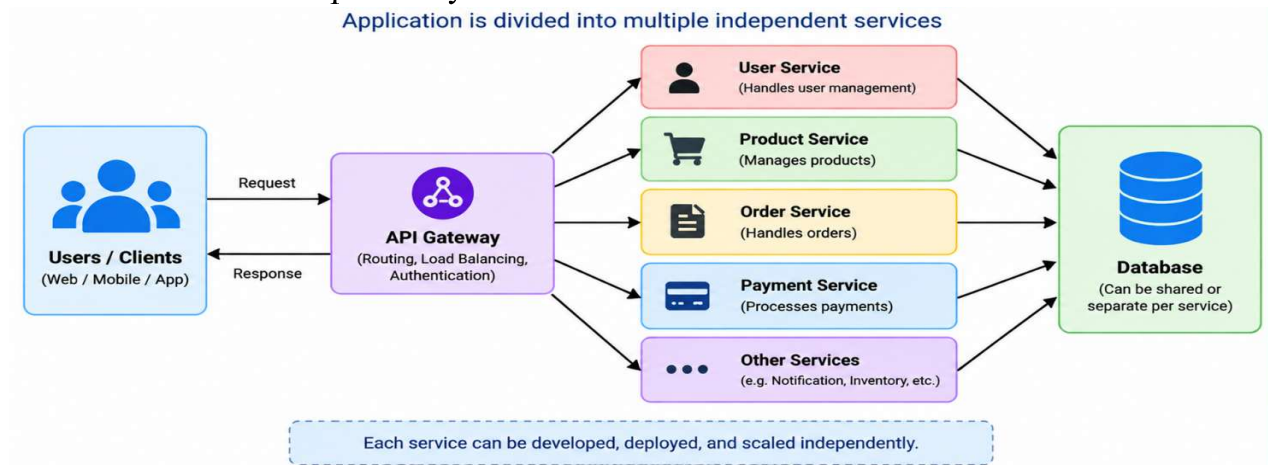
Microservices Trade-offs:

- **Advantages:** Independent scaling (e.g., scale only the Payment service during sales), fault isolation, and autonomous developer teams.

- **Disadvantages:** High operational overhead, network latency, distributed data consistency challenges, and tricky cross-service debugging.
- Clear responsibilities
- Team independence

Limitations :

- More complexity
- Network dependency



5. High-Level Design (HLD) and Low-Level Design (LLD):

High-Level Design (HLD):

High-Level Design (HLD) describes the overall architecture of a software system. It focuses on the major components of the system, how they communicate, how data flows between them, and the important architectural decisions required to meet system requirements.

HLD mainly answers: “What does the system look like, and how do its major components interact?”

HLD generally includes:

- Major components such as users, application servers, databases, caches, load balancers, and external services.
- Overall system architecture and data flow.

System Design Series

(Week 1)

- Technology choices at a high level, such as the type of database, caching system, or cloud platform.
- Major APIs and communication between components.
- Scalability strategy, such as vertical or horizontal scaling.
- Reliability and redundancy, including how Single Points of Failure can be avoided.
- Security boundaries and important external dependencies.
- Major architectural trade-offs and the reasons behind important design decisions.

Example: For a College Results Portal, an HLD may represent the system as:

Students → Load Balancer → Multiple Application Servers → Cache → Database

The HLD focuses on the overall structure of the system rather than the internal code of each component.

Low-Level Design (LLD):

Low-Level Design (LLD) focuses on the internal design of individual components. It describes how a particular component will be structured and implemented.

LLD mainly answers:

“How will each component work internally?”

LLD generally includes:

- Classes and objects.
- Attributes and methods.
- Interfaces and relationships between classes.
- Detailed business logic.
- Database tables, columns, indexes, and relationships.
- Validation and error handling.
- Detailed API request and response structures.
- Sequence flows showing how objects interact.
- Algorithms and implementation-level decisions.

System Design Series

(Week 1)

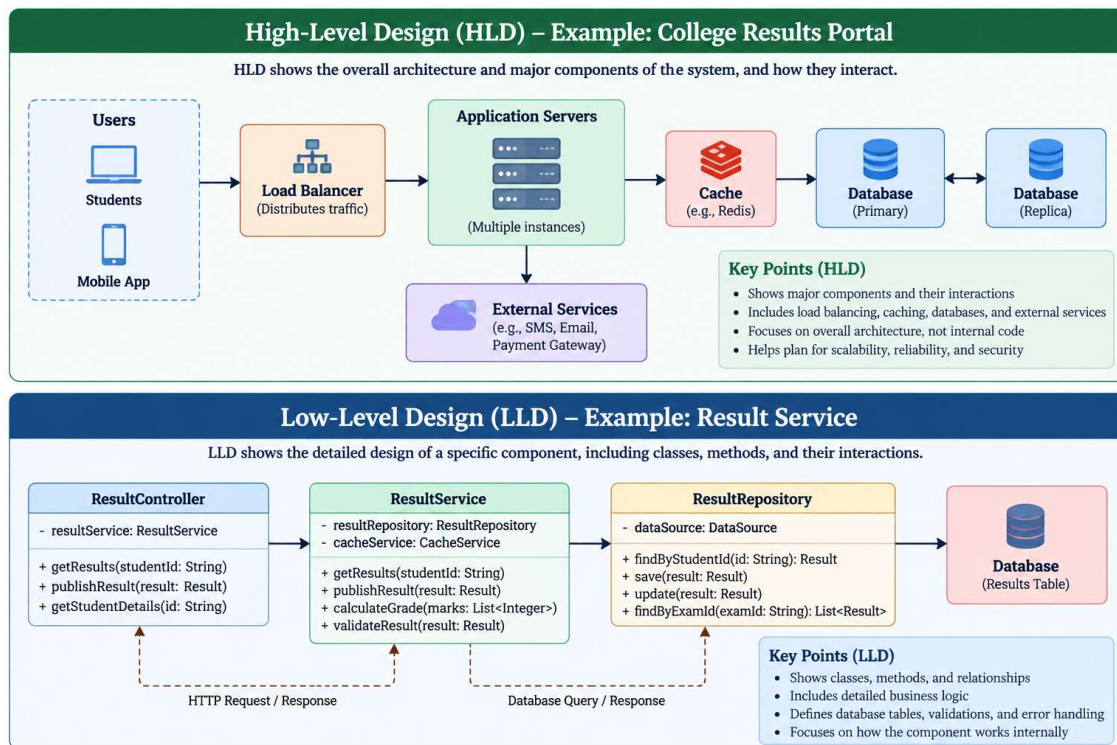
Example: If we zoom into the Result Service of the College Results Portal, the LLD may contain:

ResultController → *ResultService* → *ResultRepository* → *Database*

The ResultService may contain methods such as:

- `getResults()`
- `publishResult()`
- `calculateGrade()`
- `validateResult()`

The LLD explains what each class does and how the individual components work together internally.



6. Load Balancing: Managing Traffic Flows

A **Load Balancer** acts as a smart traffic cop positioned between users and your fleet of servers. It intercepts incoming network traffic and routes requests evenly to healthy backend servers.

- Load Balancing is the process of distributing incoming requests across multiple servers.
- A Load Balancer acts as an intermediary between users and application servers.
- It helps prevent a single server from becoming overloaded while other servers remain underutilized.

Example

Suppose an application has three servers:

- Server 1
- Server 2
- Server 3

Instead of sending every request to one server, the Load Balancer distributes requests among all three.

WorkFlow :



Core Load-Balancing Algorithms:

A load-balancing algorithm determines which server should receive an incoming request. Different algorithms are useful for different traffic patterns and server configurations.

1. Round Robin: Distributes requests sequentially (S1 → S2 → S3 → S1). Great for servers with identical hardware.

- Round Robin distributes requests sequentially across the available servers.
- Each server gets a turn before the sequence starts again.

System Design Series

(Week 1)

Example:

Request 1 → S1

Request 2 → S2

Request 3 → S3

Request 4 → S1

2. Least Connections: Sends new requests to the server with the fewest active sessions. Best when requests vary significantly in processing time.

- **Least Connections** sends a new request to the server with the fewest active connections.
- It is useful when servers have different numbers of ongoing connections.

Example :

Server	Active Connections
Server 1	10
Server 2	2
Server 3	7

The next request is sent to **Server 2** because it has the fewest active connections.

3. IP Hash: Hashes client IP addresses to direct the same user consistently to the same server (session stickiness).

- IP Hash uses the client's IP address to determine which server should receive the request.
- The same client can therefore be directed to the same server consistently, depending on the configuration.
- It can be useful when session affinity (sticky sessions) is required.

Example :

Client A → Server 1

Client B → Server 2

Client C → Server 3

4. Weighted Load Balancing: Assigns traffic weights based on server power (e.g., a 16-core server receives 3x more traffic than a 4-core server).

- Weighted Load Balancing assigns different weights to servers based on their capacity or configuration.
- A server with a higher weight receives a larger share of requests.

Example

Server 1 → Weight 5

Server 2 → Weight 3

Server 3 → Weight 2

Server 1 can therefore receive a larger proportion of the traffic.

Comparison of Load-Balancing Algorithms

Algorithm	How It Distributes Requests	Example	Best Suited For								
 Round Robin	Sends requests to servers sequentially.	Requests: S1 → S2 → S3 → S1 → S2 → ...	Servers with similar capacity and workloads.								
 Least Connections	Sends requests to the server with the fewest active connections.	<table><thead><tr><th>Server</th><th>Active Connections</th></tr></thead><tbody><tr><td>S1</td><td>10</td></tr><tr><td>S2</td><td>2</td></tr><tr><td>S3</td><td>7</td></tr></tbody></table> Next request → S2	Server	Active Connections	S1	10	S2	2	S3	7	Applications with varying connection loads.
Server	Active Connections										
S1	10										
S2	2										
S3	7										
 IP Hash	Selects a server based on the client's IP address.	Client A (IP: 10.0.0.1) → S1 Client B (IP: 10.0.0.2) → S2 Client C (IP: 10.0.0.3) → S3	Applications requiring session affinity (sticky sessions).								
 Weighted	Distributes requests according to assigned server weights.	Server Weights: S1 Weight: 5 S2 Weight: 3 S3 Weight: 2 S1 will receive a larger share of requests.	Servers with different capacities.								

7. Single Point of Failure (SPOF) & Redundancy

A **Single Point of Failure (SPOF)** is any single component in your architecture whose breakdown halts the whole system. If you have 10 app servers but only 1 database, that database is a SPOF!

- A Single Point of Failure (SPOF) is a component whose failure can cause the entire system, or an important part of it, to stop working.
- The system depends on this component because there is no alternative component available to take over its work.
- SPOFs can exist in servers, databases, network components, or other critical parts of an architecture.

System Design Series

(Week 1)

- Identifying and reducing SPOFs is an important part of designing reliable and highly available systems.

Example :

Consider an application running on only one server:

Users → Application Server → Database

If the application server fails, users cannot access the application.

Server Failure → Application Unavailable

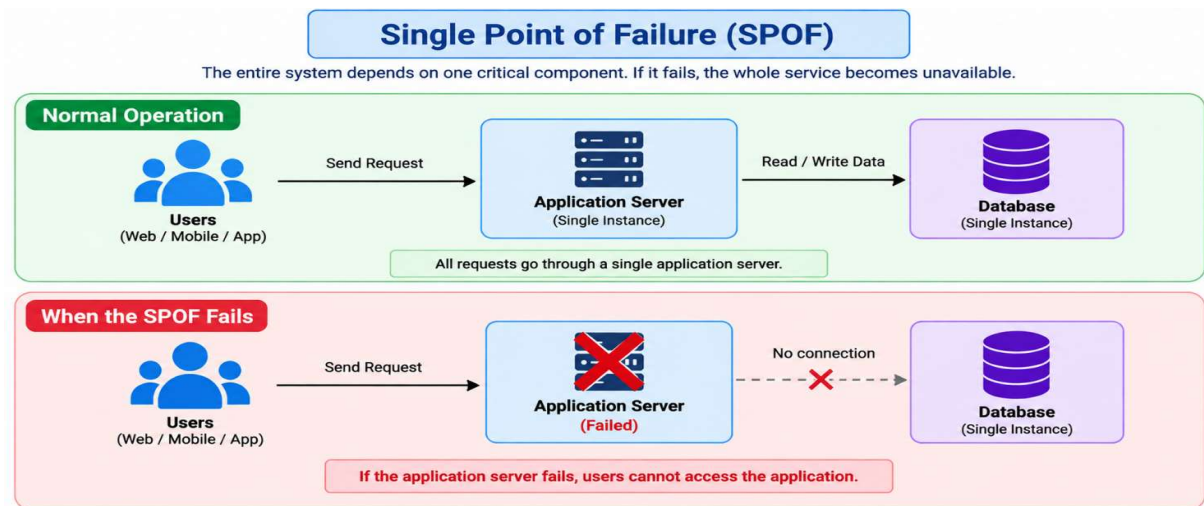
Common Examples of SPOF

1) Single Application Server

- If the only application server fails, the application becomes unavailable.
- Solution:** Use multiple application server instances.

2) Single Database

- If the only database becomes unavailable, applications depending on it may stop working. **Solution:** Use appropriate database redundancy or failover mechanisms.



Eliminating SPOFs with Redundancy & Failover

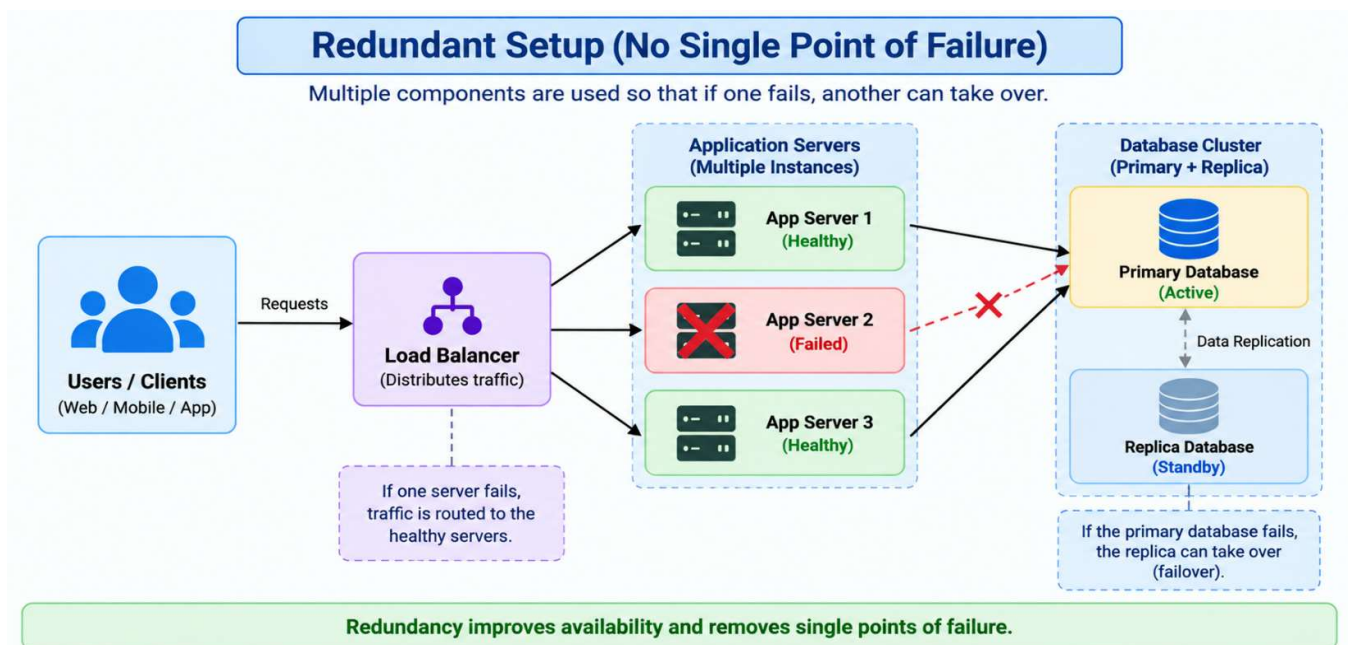
- Redundancy:** Duplicating essential components (servers, load balancers, database nodes).
- Active-Passive Setup:** Primary server processes traffic while a secondary standby node stays synced. If the primary dies, the system instantly fails over to the standby.

Users → Load Balancer → Server 1 + Server 2

If Server 1 fails, Server 2 can continue handling requests

Advantages of Redundancy:

- Reduces dependence on a single component.
- Improves system availability.
- Provides alternative capacity during failures.
- Allows maintenance with less service disruption.



8. Caching: Boosting Performance

Caching stores copies of frequently accessed data in ultra-fast RAM (like Redis or Memcached). RAM operates in nanoseconds compared to disk-based databases.

- Caching is the process of storing frequently accessed data in a faster storage layer called a cache.
- When the same data is requested again, the system can retrieve it from the cache instead of accessing the main database every time.
- This can reduce database load and improve response time for frequently requested data.

System Design Series

(Week 1)

- Caches are usually much faster to access than persistent storage such as a database.

Example:

Consider an e-commerce website where thousands of users frequently view the same product.

Instead of:

User → Application → Database → Response

the application can store frequently requested product information in a cache:

User → Application → Cache → Response

Cache Hit vs. Cache Miss

Cache Hit

- **Cache Hit:** Requested data is found in RAM cache → Instant response returned without hitting the database.
- The application retrieves the data from the cache without querying the database.

Example:

Request → Cache → Data Found → Response

Cache Miss

- **Cache Miss:** Data is missing from cache → System fetches from disk database, writes it to cache, and sends response.
- The application retrieves the data from the database and may store it in the cache for future requests.

Example:

Request → Cache → Data Not Found → Database → Cache → Response

Popular Caching Strategies

1. Cache-Aside

- **Cache-Aside (Lazy Loading):** App checks cache first. On miss, queries DB, populates cache, and returns data. Memory efficient.
- If the data is found, it is returned.

System Design Series

(Week 1)

- If not, the application retrieves it from the database and places it in the cache.

Flow:

Application → Cache → Database (if miss) → Cache

2. Write-Through

- **Write-Through:** Writes update cache and DB simultaneously. Cache is always fresh, but write latency is higher.
- The cache is updated whenever new data is written.

Flow:

Application → Cache → Database

3. Write-Behind

- **Write-Behind (Write-Back):** Writes to cache immediately and syncs to DB asynchronously. Ultra-fast writes, but risks data loss if cache crashes.
- The update to the underlying data store happens later.

Flow:

Application → Cache → Database (later)

Conclusion :

System Design provides the foundation for building software systems that can handle growth, remain reliable, and perform efficiently. By understanding scalability, architecture, load balancing, redundancy, and caching, we can design systems that adapt to changing requirements. The key is to choose the right approach based on the system's needs, constraints, and trade-offs.

System Design Series

(Week 1)

Online Resources :

Youtube Links :

- System Design Fundamentals : [System design Explained :API,Databases](#)
- Scalability & Load Balancing : [Scalability Lecture-Harvard](#)
- Monolith vs Microservices: [Monolith vs Microservices](#)
- Caching & Scalable Architecture : [System Design: Scalability Ultimate guide](#)

Practice Problems:

- Scalability & Load Balancing — [GeeksforGeeks](#)
- Caching — [GeeksforGeeks](#)
- Database Scaling & Replication — [System Design Primery](#)
- Single Point of Failure & Fault Tolerance — [GeeksforGeeks](#)

Onward to Week 2